

Python.1-4

Nome Esame: Python.1-4
Durata: 180 minuti
Punteggio Totale: 10.0 punti
Istituto: ITS - ICT Academy Lazio
Codice Esame: W78BCZNI

INFORMAZIONI STUDENTE

Nome: EDUARDO
Cognome: LEVI
Corso: CYBER

Domanda 1 (1.0 punti)**Inverti Stringa**

Scrivi una funzione che prende in input una stringa (s) e restituisce la stringa invertita.

`reverse_string(s: str) -> str`

Esempio 1:

Se `s = "ciao"`, la funzione deve restituire: `"oaic"`

Esempio 2:

Se `s = "Python"`, la funzione deve restituire: `"nohtyP"`

a) Risposta aperta - Implementare la funzione

```
def reverse_string(s: str) -> str:
    lista: list = []
    for a in s:
        lista.append(a)
    for b in lista:
        x = lista[::-1]
    return x
```


Domanda 2 (1.0 punti)**Filtra Numeri e Concatena Stringa**

Scrivi una funzione con il seguente header:

```
filter_and_concat(nums: list[int], min_val: int) -> str
```

La funzione riceve una lista di numeri interi (nums) e un valore intero (min_val).

Deve restituire una stringa contenente tutti i numeri della lista che sono **strettamente maggiori** di min_val. I numeri nella stringa risultante devono essere separati da una virgola.

Esempio:

Se nums = [10, 5, 20, 15, 3] e min_val = 12, la funzione deve restituire: "20,15"

Caso Particolare:

Se nums = [1, 2, 3] e min_val = 5, la funzione deve restituire una stringa vuota: ""

a) Risposta aperta - Implementare la funzione

```
def filter_and_concat(nums: list[int], min_val: int) -> str:
```

```
    x: str = ""
```

```
    lista = []
```

```
    for num in nums:
```

```
        if num > min_val:
```

```
            lista.append(str(num))
```

```
            if min_val not in nums:
```

```
                return x
```

```
                if x != "":
```

```
                    x += str(num)
```

```
                    x += ", "
```

```
                x += str(num)
```

```
    print(lista)
```

```
    return x
```

```
    return x
```


Domanda 3 (1.0 punti)**Conta Parole di Lunghezza Specifica**

Scrivi una funzione che riceve una lista di parole (words) e un intero (n).

La funzione deve contare e restituire quante parole nella lista hanno una lunghezza **maggiore o uguale** a n.

`count_long_words(words: list[str], n: int) -> int`

Esempio:

Se `words = ["ciao", "mondo", "python", "è", "fantastico"]` e `n = 5`:

Le parole che soddisfano il criterio sono: "mondo" (5), "python" (6), "fantastico" (10).

La funzione deve restituire: 3

Caso Particolare:

Se `words = []` (lista vuota), la funzione deve restituire: 0

a) Risposta aperta - Implementare la funzione

```
def count_long_words(words: list[str], n: int) -> int
```

```
conta    conta = 0
```

```
    for word in words:
```

```
        if len(word) >= n:
```

```
            conta += 1
```

```
    return conta
```


Domanda 4 (3.0 punti)**Sistema E-commerce**

Progetta e implementa un semplice sistema e-commerce in Python, organizzato attorno a tre classi principali: Product, ShoppingCart e Shop. Ogni classe deve essere chiara nel proprio scopo e ogni funzione deve essere ben documentata con responsabilità ben definite.

Classe Product

Rappresenta un singolo prodotto venduto nel negozio online. Ogni prodotto ha informazioni uniche e uno stock che ne regola la disponibilità.

Attributi

- `product_id: str` — Identificatore unico del prodotto nello shop (esempio: "SKU1234"). Serve per distinguere ogni articolo.
- `name: str` — Nome del prodotto (esempio: "Mouse Wireless Logitech").
- `price: float` — Prezzo unitario del prodotto in euro (ad esempio 29.90).
- `stock: int` — Quantità attualmente disponibile a magazzino per il prodotto.

Metodi

- `update_stock(quantity: int) -> None` — Aggiorna la quantità disponibile del prodotto. Se il parametro è positivo la quantità viene aumentata, se negativo viene diminuita. Non restituisce nulla.
 - `is_available(quantity: int) -> bool` — Controlla se il prodotto ha almeno la quantità richiesta disponibile (utile per le vendite). Ritorna True se lo stock è sufficiente, False altrimenti.
-

Classe ShoppingCart

Questa classe simula il carrello di un utente, che può contenere più prodotti con relative quantità. Opera come un contenitore temporaneo delle scelte d'acquisto prima del pagamento.

Attributi

- `items: dict[str, int]` — Dizionario che associa ogni `product_id` alla quantità

richiesta da acquistare da parte dell'utente. Esempio: { 'SKU1234': 2, 'SKU5678': 1 }.

Metodi

- `add_item(product: Product, quantity: int) -> str` — Tenta di aggiungere il prodotto al carrello nella quantità richiesta. Se la quantità disponibile non è sufficiente, restituisce un messaggio di errore. Se l'aggiunta va a buon fine, aggiorna lo stock del prodotto (diminuisce di `quantity`) e ritorna un messaggio di conferma.
- `remove_item(product_id: str) -> str` — Rimuove dal carrello il prodotto corrispondente al `product_id`. Se il prodotto era presente, lo elimina e restituisce conferma; se non era presente, restituisce un messaggio di errore.
- `get_total(products: dict[str, Product]) -> float` — Calcola e ritorna la somma totale degli articoli nel carrello, moltiplicando il prezzo unitario per la quantità di ogni prodotto. Ha bisogno del dizionario dei prodotti per accedere ai prezzi.
- `clear()` -> `None` — Svuota tutto il carrello, rimuovendo tutti i prodotti e le relative quantità.

Classe Shop

Gestisce l'inventario dei prodotti e tutti i carrelli attivi (potrebbero corrispondere a diversi utenti o sessioni). Offre metodi per la gestione dei prodotti, la creazione di carrelli e la conclusione dell'acquisto (checkout).

Attributi

- `products: dict[str, Product]` — Mappa ogni `product_id` a un oggetto `Product`, rappresentando l'intero inventario del negozio.
- `carts: dict[str, ShoppingCart]` — Mappa ogni carrello (identificato da un `cart_id`, es: "cart123") all'oggetto `ShoppingCart` corrispondente. Così si gestiscono più clienti contemporaneamente.

Metodi

- `add_product(product: Product) -> str` — Aggiunge un nuovo prodotto al negozio. Se il `product_id` è già presente, non aggiunge il prodotto e restituisce un messaggio di errore; altrimenti, aggiunge il nuovo prodotto e ritorna una stringa di conferma.
- `remove_product(product_id: str) -> str` — Rimuove il prodotto col dato `product_id` dall'inventario. Restituisce conferma se l'operazione ha successo o un errore se il prodotto non esiste.
- `create_cart()` -> `str` — Crea un nuovo carrello della spesa, genera un nuovo `cart_id` (puoi usare ad esempio `uuid`), lo inserisce nella mappa `carts` e ritorna il suo id.
- `checkout(cart_id: str) -> float | str` — Conclude l'acquisto per il carrello identificato da `cart_id`. Se tutti i prodotti nel carrello sono effettivamente disponibili negli

Class Product:

```
def __init__(self, product_id: str, name: str, price: float, stock: int):
    self.product_id = product_id
    self.name = name
    self.price = price
    self.stock = stock
```

```
def update_stock(self, quantity: int) -> None:
    self quantity += quantity
```

```
def is_available(self, quantity: int) -> bool:
    if self.stock >= quantity:
        return True
    else:
        return False
```

Class ShoppingCart:

```
def __init__(self, items: dict[str, int] = {}):
    self.items = items
```

```
def add_item(self, product: Product, quantity: int) -> str:
    if product not product.is_available():
        return "Error"
    else:
        return "Conferma"
```




```
def remove_item(self, product_id: str) -> str:
```

```
    if not product_id in self.product.keys():
```

```
        self.product[product_id].remove(product_id)
```

```
        return "Prod. Eliminato"
```

```
    else:
```

```
        return "Error"
```

```
def get_total(self, products: dict[str, Product]) -> float:
```

```
    Somma_ord: float = 0
```

```
    for product_id, quantity in self.items.items():
```

```
        if product_id in products:
```

```
            Somma_ord += product[product_id].price * quantity
```

```
    return Somma_ord
```

```
def clear(self) -> None:
```

```
self.products.pop(Product)
```

```
self.items.pop
```


class Shop:

ERRORS LEN

```
def __init__(self, product: dict [str, Product], carts: dict [str, ShoppingCart]):
```

```
    self.product = product
```

```
    self.carts = carts
```

```
def add_Product (self, product: Product) -> str
```

```
if product.Product_id in self
```

```
if product.Product_id in self.Products:  
    return "Error"
```

```
else:
```

```
    self.product[product.Product_id] = product
```

```
def remove_Product(Product_id) -> str:
```

```
if Product_id not in self.Products:  
    return "Error"
```

```
else:
```

```
    del self.Product[Product_id]
```

```
def create_cart(self):
```

```
    cart_id = str(uuid.uuid4())
```

```
    self.carts[cart_id] = {}
```

```
    return cart_id
```

```
def checkout(self, cart_id: str) -> float | str:
```

```
if cart_id not in self.carts:  
    return "Error"
```

```
cart = self.carts[cart_id]
```

```
if not cart:
```

```
    return "ERROR"
```

```
total = 0.0
```

```
for a, b in cart.items():
```

```
    if a not in self.product:
```

```
        return "ERROR"
```

```
    Product = self.product[a]
```



```
if prodotto.quantity < b:
```

```
    return "Errore"
```

```
totale += prodotto.Price * b
```

```
for a, b carts.item():
```

```
    self.product[a].quantity -= b
```

```
self.carts[cart_id] = {}
```

```
return totale
```



stock, calcola il totale, aggiorna definitivamente le giacenze e svuota il carrello. Se manca disponibilità per uno o più prodotti durante questa verifica, restituisce un messaggio di errore (ad esempio specificando quali oggetti non sono disponibili) e non conclude l'acquisto.

Nota Bene

- Ogni metodo dovrebbe controllare con precisione la validità dell'operazione richiesta e fornire messaggi chiari in caso di errore.
- Si assuma che ogni `cart_id` e `product_id` sia univoco sul sistema.
- L'esercizio valuterà la correttezza dell'organizzazione delle classi e delle responsabilità di ogni metodo.

a) Risposta aperta - Implementare le tre classi complete

Domanda 5 (1.0 punti)**Gestione Parcheggio**

Progetta una classe `ParkingLot` per la gestione dei posti auto (spazi) in un parcheggio. Ogni posto è identificato da un numero intero.

Classe `ParkingLot`**Attributi**

- `spaces: dict[int, dict]`: Dizionario che mappa il numero del posto (intero) ai dettagli del posto. I dettagli sono un dizionario con: `"occupato": bool` e `"targa": str` | `None`.

Metodi

- `add_space(numero: int) -> bool`
Controlla se `numero` (il posto) esiste già in `spaces`. Se sì, restituisce `False`. Altrimenti, aggiunge il nuovo posto (impostandolo come `occupato=False` e `targa=None`) e restituisce `True`.
- `remove_space(numero: int) -> dict | str`
Cerca `numero` in `spaces`. Se non esiste, restituisce `"Errore: posto inesistente."`. Se esiste ma è `occupato=True`, restituisce `"Errore: posto occupato."`. Se esiste ed è `occupato=False`, lo rimuove da `spaces` e restituisce un dizionario contenente *solo* i dettagli del posto rimosso.
- `park_car(numero: int, targa: str) -> str`
Cerca `numero` in `spaces`. Se non esiste, restituisce `"Errore: posto inesistente."`. Se esiste ma è `occupato=True`, restituisce `"Errore: posto già occupato."`. Se esiste ed è libero, imposta `occupato=True`, salva la `targa` e restituisce `"Auto parcheggiata con successo."`.
- `leave_space(numero: int) -> str`
Cerca `numero` in `spaces`. Se non esiste, restituisce `"Errore: posto inesistente."`. Se esiste ma è `occupato=False`, restituisce `"Errore: posto già libero."`. Se esiste ed è occupato, imposta `occupato=False` e `targa=None`, e restituisce `"Posto liberato con successo."`.
- `list_free_spaces() -> list[int]`
Scorre tutti i posti in `spaces` e restituisce una lista contenente i numeri (le chiavi) di *solo* quei posti che hanno `occupato=False`.
- `get_space_info(numero: int) -> dict | str`
Cerca `numero` in `spaces`. Se non esiste, restituisce `"Errore: posto inesistente."`.

Altrimenti, restituisce il dizionario *interno* contenente i dettagli di quel posto (stato di occupazione e targa). (es. {"occupato": True, "targa": "AB123CD"}).

a) Risposta aperta - Implementare la classe completa

Domanda 6 (0.2 punti)

Come si gestisce un'eccezione specifica?

- a) handle ValueError:
- b) except (ValueError):
- c) catch ValueError:
- ☒ d) except ValueError:

Domanda 7 (0.2 punti)

Cosa restituisce type(42) in Python?

- ☒ a) int
- b) number
- c) 42
- d)

Domanda 8 (0.2 punti)

Quale delle seguenti è una caratteristica delle liste in Python?

- a) Le liste sono immutabili.
- b) Le liste non possono contenere elementi duplicati.
- ☒ c) Le liste sono ordinate.
- d) Le liste non supportano tipi misti.

Domanda 9 (0.2 punti)

Come si definisce una classe in Python?

- a) object NomeClasse:
- b) class NomeClasse {
- c) def class NomeClasse:
- ☒ d) class NomeClasse:

Domanda 10 (0.2 punti)

Come si apre un file in modalità lettura?

- ☒ a) with open('file.txt', 'r') as file:
- b) open('file.txt', 'input')
- c) open('file.txt', 'readonly')
- d) open('file.txt', 'read')

Domanda 11 (0.2 punti)

Quale eccezione si solleva quando si divide per zero?

- a) DivisionError
- b) MathError
- c) ArithmeticError
- ☒ d) ZeroDivisionError

Domanda 12 (0.2 punti)

Quale operatore si usa per l'assegnazione in Python?

- ☒ a) =
- b) ==
- c) !=
- d) :=

Domanda 13 (0.2 punti)

Come si gestisce un'eccezione in Python?

- ☒ a) try: ... except:
- b) try: ... rescue:
- c) try: ... handle:
- d) try: ... catch:

Domanda 14 (0.2 punti)

Come si rimuove una chiave da un dizionario?

- ☒ a) dict.remove('chiave')
- b) dict.delete('chiave')
- ☒ c) del dict['chiave']
- d) dict.del('chiave')

Domanda 15 (0.2 punti)

Quale metodo si usa per convertire una stringa in minuscolo?

- a) down()
- b) small()
- ☒ c) lower()
- d) toLowerCase()

Domanda 16 (0.2 punti)

Come si salta all'iterazione successiva in un ciclo?

- a) skip
- ☒ b) continue
- c) jump
- d) next

Domanda 17 (0.2 punti)

Come si esce da un ciclo for in Python senza ritornare nessun valore?

- a) exit
- b) stop
- ☒ c) break
- d) return

Domanda 18 (0.2 punti)

Quali sono le modalità di apertura dei file in Python?

- a) Open, close, delete
- b) Json, csv, xml
- ☒ c) "r", "w", "a"
- d) Text, binary, append

Domanda 19 (0.2 punti)

Come si itera su un dizionario in Python?

- ☒ a) for chiave, valore in dict.items():
- b) for (chiave, valore) in dict:
- c) for chiave in dict:
- d) foreach chiave, valore in dict:

Domanda 20 (0.2 punti)

Come si chiude un file?

- a) file.terminate()
- ☒ b) file.close()
- c) file.finish()
- d) file.end()